Process & Decision Documentation

Side Quests and A4 (Individual Work)

- Changed background colour to bright green to evoke sense of “panic”; it definitely made me feel panicked and uneasy

Role-Based Process Evidence

Name: Joanne Lang

Goal of Work Session

- Redesigned the blob’s movement and environment to express panic
    - Make the blob’s movement more erratic
    - Change the background to an anxious green colour

Tools, Resources, or Inputs Used

- GenAI tools: ChatGPT
- Lecture Notes: GBDA302 Winter 2026 Starter Guide
- Teammates: N/A
- Prior drafts or code: GBDA302_W2_codeExamples
- Playtesting feedback

GenAI Documentation

Date Used: Thurs. Jan 22

Tool Disclosure: ChatGPT-5 mini

Purpose of Use: brainstorming

Summary of Interaction: identified which line of code changed the background colour; gave me the RGB code for bright green; informed me of the version of ChatGPT I was using

Human Decision Point(s): Ignored its suggestions to make the background greyscale or different colours.

Integrity & Verification Note: Tried changing the background values in my own code, saw that it worked and was accurate.

Scope of GenAI Use: I changed the visual properties and breathing animation of my blob on my own.

Limitations or Misfires: n/a, didn't ask much of it so the answers it gave were good; no limitations/misfires here

Summary of Process (Human + Tool)

Describe what you did, focusing on process rather than outcome. This may include:

- Prompted how to change my background colour; I had to sift through the many options it gave me of different colours to understand where that option in my code is.
- Didn't know the RGB code for bright green, so asked Chat
- Verified which version I was using by asking it itself

Decision Points & Trade-offs

Describe one or two key decisions you made:

- Considered using one of the RGB colour codes it suggested; decided I still wanted to make background bright green to make the feel of the game more "panic"

Verification & Judgement

- I looked at the result and it did make me feel more panicked

Limitations, Dead Ends, or Open Questions

- N/A

Appendix

given the following code, how would i change the background colour: // Y-position of the floor (ground level) let floorY3; // Player character (soft, animated blob) let blob3 = { // Position (centre of the blob) x: 80, y: 0, // Visual properties r: 26, // Base radius points: 7, // Number of points used to draw the blob wobble: 24, // Edge deformation amount wobbleFreq: 1.2, // Time values for breathing animation t: 0, tSpeed: 0.75, // Physics: velocity vx: 0, // Horizontal velocity vy: 0, // Vertical velocity // Movement tuning accel: 0.55, // Horizontal acceleration maxRun: 4.0, // Maximum horizontal speed gravity: 0.65, // Downward force jumpV: -11.0, // Initial jump impulse // State onGround: false, // True when standing on a platform // Friction frictionAir: 0.995, // Light friction in air frictionGround: 0.88, // Stronger friction on ground }; // List of solid platforms the blob can stand on // Each platform is an axis-aligned rectangle (AABB) let platforms = []; function setup() {

createCanvas(640, 360); // Define the floor height floorY3 = height - 36; noStroke(); textFont("sans-serif"); textSize(14); // Create platforms (floor + steps) platforms = [{ x: 0, y: floorY3, w: width, h: height - floorY3 }, // floor { x: 120, y: floorY3 - 70, w: 120, h: 12 }, // low step { x: 300, y: floorY3 - 120, w: 90, h: 12 }, // mid step { x: 440, y: floorY3 - 180, w: 130, h: 12 }, // high step { x: 520, y: floorY3 - 70, w: 90, h: 12 }, // return ramp]; // Start the blob resting on the floor blob3.y = floorY3 - blob3.r - 1; } function draw() { background(240); // --- Draw all platforms --- fill(200); for (const p of platforms) { rect(p.x, p.y, p.w, p.h); } // --- Input: left/right movement --- let move = 0; if (keyIsDown(65) || keyIsDown(LEFT_ARROW)) move -= 1; // A or ← if (keyIsDown(68) || keyIsDown(RIGHT_ARROW)) move += 1; // D or → blob3.vx += blob3.accel * move; // --- Apply friction and clamp speed --- blob3.vx *= blob3.onGround ? blob3.frictionGround : blob3.frictionAir; blob3.vx = constrain(blob3.vx, -blob3.maxRun, blob3.maxRun); // --- Apply gravity --- blob3.vy += blob3.gravity; // --- Collision representation --- // We collide using a rectangle (AABB), // even though the blob is drawn as a circle let box = { x: blob3.x - blob3.r, y: blob3.y - blob3.r, w: blob3.r * 2, h: blob3.r * 2, }; // --- STEP 1: Move horizontally, then resolve X collisions --- box.x += blob3.vx; for (const s of platforms) { if (overlap(box, s)) { if (blob3.vx > 0) { // Moving right → hit the left side of a platform box.x = s.x - box.w; } else if (blob3.vx < 0) { // Moving left → hit the right side of a platform box.x = s.x + s.w; } blob3.vx = 0; } } // --- STEP 2: Move vertically, then resolve Y collisions --- box.y += blob3.vy; blob3.onGround = false; for (const s of platforms) { if (overlap(box, s)) { if (blob3.vy > 0) { // Falling → land on top of a platform box.y = s.y - box.h; blob3.vy = 0; blob3.onGround = true; } else if (blob3.vy < 0) { // Rising → hit the underside of a platform box.y = s.y + s.h; blob3.vy = 0; } } } // --- Convert collision box back to blob centre --- blob3.x = box.x + box.w / 2; blob3.y = box.y + box.h / 2; // Keep blob inside the canvas horizontally blob3.x = constrain(blob3.x, blob3.r, width - blob3.r); // --- Draw the animated blob --- blob3.t += blob3.tSpeed; drawBlobCircle(blob3); // --- HUD --- fill(0); text("Move: A/D or ←/→ • Jump: Space/W/↑ • Land on platforms", 10, 18); } // Axis-Aligned Bounding Box (AABB) overlap test // Returns true if rectangles a and b intersect function overlap(a, b) { return (a.x < b.x + b.w && a.x + a.w > b.x && a.y < b.y + b.h && a.y + a.h > b.y); } // Draws the blob using Perlin noise for a soft, breathing effect function drawBlobCircle(b) { fill(20, 120, 255); beginShape(); for (let i = 0; i < b.points; i++) { const a = (i / b.points) * TAU; // Noise-based radius offset const n = noise(cos(a) * b.wobbleFreq + 100, sin(a) * b.wobbleFreq + 100, b.t,); const r = b.r + map(n, 0, 1, -b.wobble, b.wobble); vertex(b.x + cos(a) * r, b.y + sin(a) * r); } endShape(CLOSE); } // Jump input (only allowed when grounded) function keyPressed() { if ((key === " " || key === "W" || key === "w" || keyCode === UP_ARROW) && blob3.onGround) { blob3.vy = blob3.jumpV; blob3.onGround = false; } } /* In-class tweaks for experimentation: • Add a new platform: platforms.push({ x: 220, y: floorY3 - 150, w: 80, h: 12 }); • “Ice” feel → frictionGround = 0.95 • “Sand” feel → frictionGround = 0.80 */

You're already *very* close — the background colour is controlled by **one line** in your `draw()` loop 👍

Where the background is set

This line does it:

```
background(240);
```

In p5.js, `background()` clears the canvas every frame and fills it with a colour.

How to change the background colour

1. Grayscale

```
background(0);    // black
background(255);  // white
background(180);  // light gray
```

---

2. RGB colour

```
background(135, 206, 235); // sky blue
background(30, 30, 40);    // dark/night vibe
background(255, 240, 220); // warm cream
```

Format:

```
background(red, green, blue);
```

Each value is `0–255`.

---

3. RGBA (with transparency)

Useful if you *don't* want a full clear every frame (for trails, motion blur effects):

```
background(240, 240, 240, 50);
```

⚠️ This only works if you want visual accumulation — otherwise stick to opaque.

Example: soft sky + darker platforms

```
function draw() {
  background(220, 235, 255); // light sky blue

  fill(180); // platforms
  for (const p of platforms) {
    rect(p.x, p.y, p.w, p.h);
  }

  ...
}
```

Bonus: dynamic background (based on height, time, etc.)

For example, changing colour as the blob moves up:

```
let bg = map(blob3.y, 0, height, 200, 240);
background(bg, bg + 10, 255);
```

If you want:

- 🌅 parallax sky
- 🌙 day/night cycle
- 🎮 gradient background
- 🌊 animated background

tell me the vibe and I'll wire it directly into *this* sketch.

what should i enter to make it bright green

Use this 👇

```
background(0, 255, 0);
```

That's **pure bright green** in RGB.

If you want a couple nice variations:

```
background(50, 255, 50);   // slightly softer bright green
background(0, 200, 0);     // darker green
background(0, 255, 80);    // neon-ish green
```

Drop it straight into your `draw()` function where `background(240);` is now.

what version of chatgpt am i currently using

You're using **GPT-5 mini**.